CANDY
DIALOGUE ENGINE

Variants & Translations

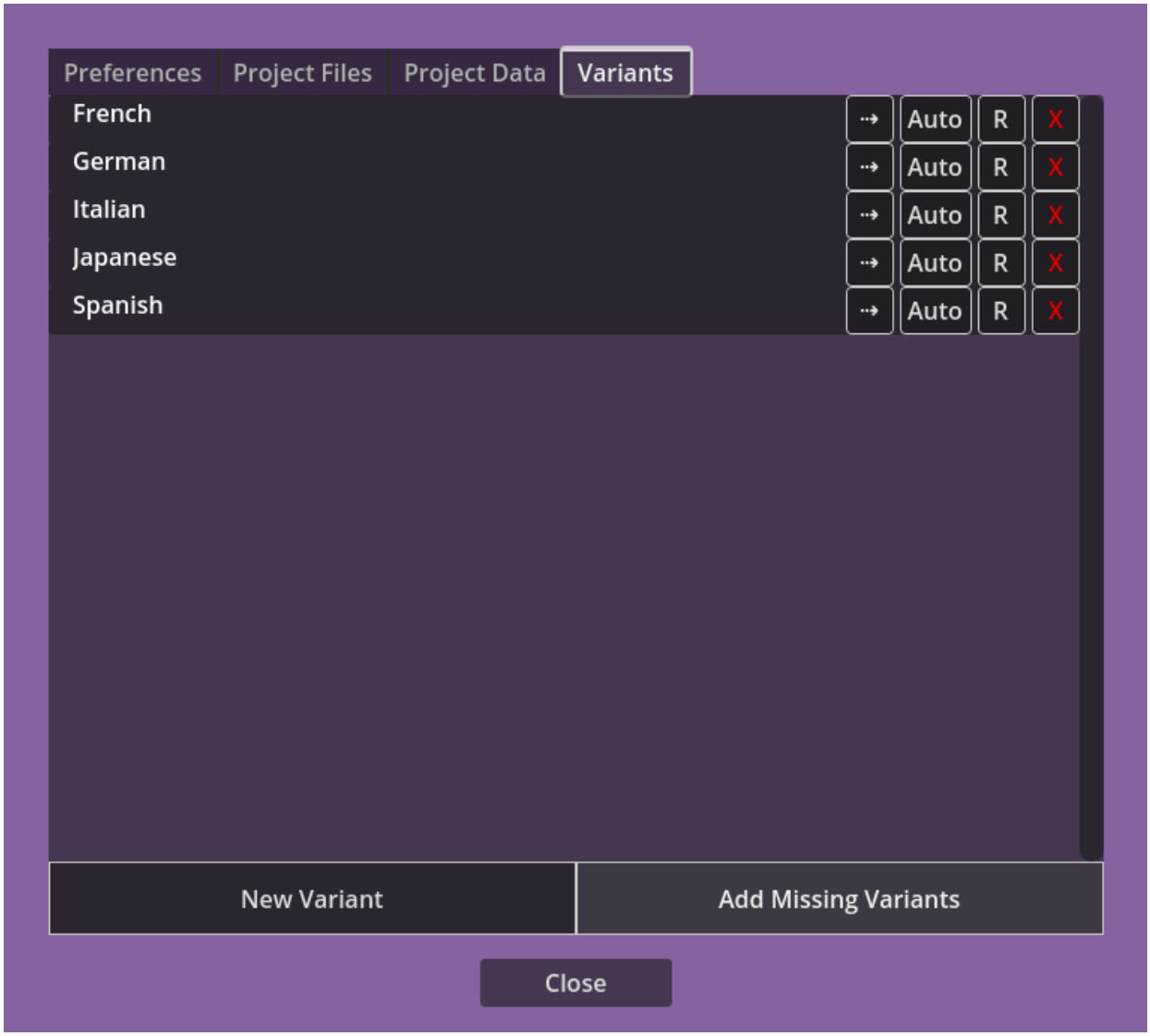
1.0.0

What should you do if you want to translate your dialogues to other languages, or create alternate texts for a same spoken line? Candy DE offers an elegant solution: variants.

Variants are alternate text lines attached to a spoken line. In fact, a spoken line's original text is actually a variant named "Default".

Creating Variants

In the Candy Dialogue Creator, open the main menu (≡ button in the top-left corner), click "Settings", then click the "Variants" tab:



Click "New Variant" at the bottom, and give the new variant a name of your choosing. Then, click "Confirm".

This will add a new variant to your profile in Candy DC.

You can configure a couple of things:

- If the language for this variant is written right-to-left, click the "⇢" button to toggle it.
- If this variant should NOT be automatically added to new spoken lines, click the "Auto" button to deactivate it.

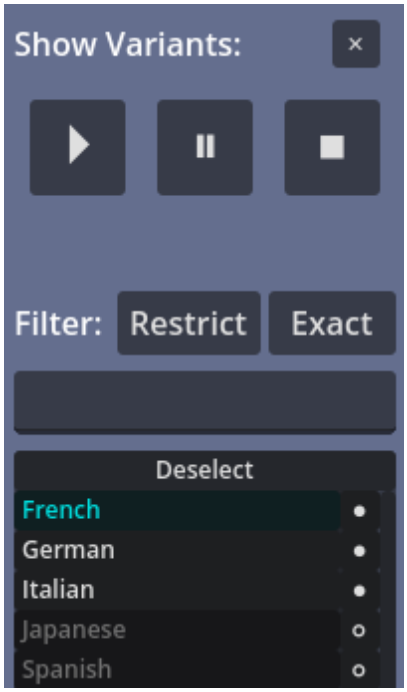
The "R" button is used for renaming a variant, and will update all the lines in the currently opened dialogue to switch to the new name.

The red 'X' button deletes a variant, and will delete the variant from all lines that use it.

Writer UI

Access the Writer by adding a Spoken Line and click the "⇱", button in that line.

In the Writer, tick the "Show Variants" box to display the variant controls.



Variant List

Notice the list of variants: those in white are the variants enabled for the selected line, while the variants in grey are disabled. Disabled variants don't exists for the particular line.

You can toggle enabled/disabled by clicking the dot next to each variant. Disabling a variant will delete it from the line, and any text you wrote for it will be lost.

To select a variant, just click its name. It will become colored cyan.

Filters

You can easily filter variants to make the list display only those you're interested in.

In the filter text field, type one or more words separated by commas (,), then toggle the buttons above:

"Restrict" will only show variants that match at least one of the words, while "Exclude" will show variants that don't match any words.

"Exact" will only show or hide variants whose name is an exact match to one or more of the filter words, and "Loose" will show or hide variants who have at least one of the words in their names.

As an example, if you type "French, Spanish" as filter words and set the filters to "Restrict" and "Exact", only the variants named "French" and "Spanish" will show in the variant selection list (if they exist).

If you set the filters to "Loose" instead of "Exact", variants such as "French_1, French_2", and so on will also show in the list.

Writing Area

When you enable the "Show Variants" button, two new fields appear in the writing area to the right: a writing field and a preview field for the variant.

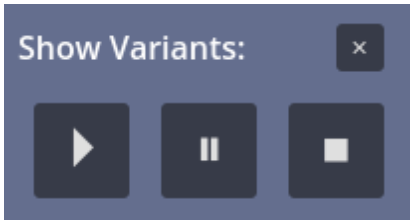
This lets you write the text for the selected variant while being able to look at the original text. And it also lets you compare the previews for both, for example to verify that the BBCode formatting is the same as the original.

The variant writing area has a text direction button (⇢). Toggling it will change the direction for the selected line, for the selected variant. You might use this if a variant's normal text direction is left-to-right, but one specific line appears in another language and should be written right_to_left (or the other way around).

The variant writing area also displays a character and word count: this indicates how many more/less words the variant has compared to the original.

Voice Preview

You can preview the voice file for the selected variant (if it has one) by using the controls under the "Show Variants" button:



At this time, the volume and progress sliders are shared with those of the original voice file.

Uses

In-game display

Variants have several uses, although you might find more beyond what Candy DE anticipated. As you will see, it's a highly flexible feature, so we're certain some developers will get very innovative.

To display variants in your game, set the 'language' variable in Candy_Settings.gd to the name of any variant. For translations, you might want to let players change this value themselves through an option menu, so write a function that does so (use 'candy.language' to refer to the variable).

When a spoken line is displayed on screen, the text of the chosen variant will be shown. If a line doesn't have a matching variant, the original text will be shown instead.

Remember that the original text is always saved to a variant named "Default". So if you write your original dialogues in English, you would set language = "Default", not language = "English".

Translations

The most obvious use for variants is translating dialogues into multiple languages, for games that want to provide this feature. Just name a variant after any language (e.g. "French"), and write a translation of the original text for it. That's all there is to it!

Moral Routes, Character Classes, Other Fun Stuff

You don't have to use variants only for translations: you can also use them for moral routes (e.g. "Good" playthrough vs. "Evil" playthrough), character classes (different speech for warrior, wizard, rogue...) or anything else you can imagine.

Just create variants with any name (e.g. "Good" or "Warrior") and assign the appropriate value to candy.languages at runtime.

You can also mix translations, moral routes, classes any anything else. For example, you can create "Default_Good", "Default_Evil", "French_Good", French_Evil" variants, or "Default_Good_Warrior", "Default_Good_Wizard", and so on...

Of course, you'll need to write a small function that combines all these variables into a single correct variant name to assign to the candy.language variable, but that's trivial:

candy.language = chosen_language + "_" + moral_route + "_" + player_class

Gendered Variations

Candy DE also provides a way to create variants based on the gender of the speaker, the player or both.

To make a variant match the gender of the speaker, add the "_S:" suffix followed by the gender you want for the given variant. For example, "Default_S:F" will display this variant if the selected

language is "Default" and if the actor speaking the line has "F" as their gender in the actors_dictionary.

To match a variant to the player's gender instead, the method is similar: use the "_P:" suffix. For example: "Default_P:M".

Note that "_P:" will match the player's gender not inside actors_dictionary, but with the 'candy_ui.player_gender' variable in Candy_Settings.gd. This is where you must write or update the player's gender to make the feature work properly.

That being said, Candy DE offers other tools and methods for adapting speech based on gender. This is one option, but others might be more practical or better fit your own preferences. See the Word Substitution guide for instance.

Random Variations

Variants can also be used to create variations for a same line and language. For example, if NPCs frequently repeat some lines, you may want to change things a bit to avoid annoying repetitions for the player.

Candy DE has a built-in feature for this: just create some new variants named "Default_#1", "Default_#2" and so on. Basically, add the '_#n' suffix, where 'n' is any number. Numbers don't need to be sequential, so "Default_#1, "Default_#5" will work even if #2, #3 and #4 are missing.

When Candy DE processes a spoken line, it checks for variants that have these suffix. If it finds them, it chooses one at random. Note that it can also choose the original variant at random (i.e. "Default").

This works for other languages too: if you set language = "French", Candy DE will randomly choose among "French", "French_#1", "French_#5" and so on (whatever variants for random variation exist for a line).

Note that if a variant with other suffixes is selected before the random selection step (e.g. "French_S:F_P:M"), the engine will select from the random variants and the last non-random variant (i.e. if the selected non-random variant was "French_S:F_P:M", the engine will select from that variant and the random variant, but won't include "French" and "French_S:F" as a possible random choice).

Variants can be assigned a weight, which determines their odds of being chosen (this includes non-random variants). Weights are relative to one another: for example, if Default_#1 has a weight of 10, and Default_#2 has a weight of 20, then Default_#2 has twice as many odds of being selected than Default_#1.

Weights are assigned in Candy DC's Writer UI, not in the variant name. If you ever want to ensure a variant has no odds of being randomly selected, assign it a weight of 0.

Chaining Suffixes

Okay, we're going to get a little bit technical here. But don't worry, because this is much simpler than it looks at first glance. That wall of text that follows? It's there to tell you "See? You don't need to know anything!", not "You need to know all this!" -- Candy DE basically holds your hand here.

You're not restricted to only one suffix. For example, you can create a variant named "French_P:F_S:M_#1". This will display if the player's gender is "F", the speaker's gender is "M", and if variant #1 is drawn at random among other "French_P:F_S:M" variants.

One important detail: **suffix order matters!** That's the one thing you must remember.

Player Gender ("_P:") first, Speaker Gender ("_S:") second, and Random ("_#n") last:

Default_P:M_S:F_#1

This is because of the order of operations:

1) The engine first looks for a variant that starts with the value of candy.language (e.g. "French"). If it doesn't find one, it continues with "Default" instead.
2) It then checks the player's gender. If a gender is specified for the player (e.g. candy_ui.player_gender = "M") it now looks for any variant starting with "French_P:M".
3) It then looks at the speaker's gender (e.g. "F"):
    a) If variants starting with "French_P:M" were found earlier, it looks for variants starting with "French_P:M_S:F"
    b) If no variants starting with "French_P:M" were found earlier, it looks for variants starting with "French_S:F" instead.
4) Finally, it looks for matching variants that end with the "_#n" suffix.
    a) If it found "French_P:M_S:F" at 3a earlier, it looks for any "French_P:M_S:F_#n".
    b) If it found only "French_S:F" at 3b earlier, it looks for any "French_S:F_#n".
    c) If it found neither, it looks for any "French_#n".

In other words, **you never have to concern yourself with missing variants or genders**. When no suitable variant is found, or if the player or speaker's gender isn't specified, Candy DE figures out the correct alternatives. The worst that can happen if you name variants wrong or forgot to include a variant, is that Candy DE will display the original text instead ("Default").

To keep it simple: just name variants after the genders they must match (and make sure actor and player genders are correctly defined).

Does the text change depending on player gender? Add "_P:".
Does the text change depending on speaker gender? Add "_S:".

Does it change depending on both? Add both!
Do you want randomness too? Add "_#n", optionally with a weight: "_#n%x"

This is all there is to it.